.NET Frontend Technologies Compared

Razor Pages, Blazor Server and Blazor WebAssembly with SimpleStudent examples

Module: Heterogeneous Systems
Group: PSe-5
Autumn 2026
Lecturer: Tadas Bulis
Kauno kolegija, Faculty of Technologies

Contents

1. Purpose
2. Three frontends in our system
3. How each technology works
4. Comparison table
5. The same task in three ways
6. CORS: why only WebAssembly needs it
7. When to choose which
8. Common problems and fixes
9. Running the projects
10. Practical tasks
11. References

1. Purpose

In our heterogeneous system the same Students API (.NET, PHP and Python versions) has several clients. Three of them are written in .NET: **Razor Pages**, **Blazor Server** and **Blazor WebAssembly**. All three show the same data and use the same client generated from `swagger.json` (`IStudentApiClient`), yet they work in fundamentally different ways.

This guide explains, for each technology, **where the C# code runs**, **who calls the API** and **how the page is updated**. All code samples are taken from the course projects in the folder `student-docker-lab-plus-mssql`, so every statement can be checked by running the projects.

> **Key idea.** Razor Pages and Blazor are **frontend** technologies. Neither of them replaces the API. They are different ways to build a user interface that talks to the same backend over HTTP and JSON.

2. Three frontends in our system

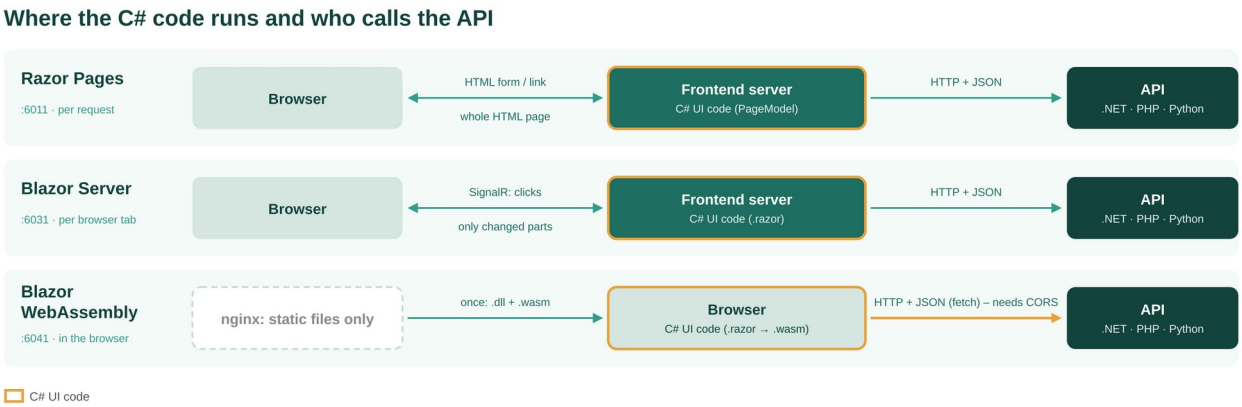


Figure 1. One API, three .NET clients. The orange frame shows where the C# UI code runs.

Project	Technology	Port	What to notice
SimpleStudentFrontend	Razor Pages	6011	Every action is an HTML form or a link, after which the browser reloads the whole page.
SimpleStudentBlazorServer	Blazor Web App, Interactive Server	6031	No page reloads; API switcher; live search; validation while typing.
SimpleStudentBlazorWasm	Blazor WebAssembly (standalone) + nginx	6041	The same components, but running in the browser; F12 → Network shows the requests to the API.

In the Blazor projects the backend (.NET, PHP or Python) can be chosen with the buttons at the top right. All three APIs implement the same contract, so one generated client is enough for all of them.

3. How each technology works

3.1 Razor Pages

Razor Pages is the classic server-side model: every page has a `.cshtml` template and a `PageModel` class (`.cshtml.cs`). Every request stands on its own:

1. The browser sends `GET /Students` (or `POST` when a form is submitted).
2. The server creates `StudentsModel` and calls `OnGetAsync()`, which calls the API through `IStudentApiClient`.
3. The server renders the **whole HTML page** and sends it to the browser. The `PageModel` object is discarded.
4. Any action (delete, save) is again a new request and a new page (often via `RedirectToPage()`).

3.2 Blazor Server (Interactive Server)

A Blazor UI is made of **components** (`.razor` files) that contain both HTML and C# code (`@code { }`). In Interactive Server mode the components run on the server:

1. The browser loads the page and `blazor.web.js`, which opens a **SignalR** (WebSocket) connection to the server.
2. The server keeps state for every browser tab, called a **circuit**. The components and their fields (`_students`, `_search`) live in server memory.

3. A click or key press is sent over SignalR. The server runs the C# method (e.g. `DeleteAsync`), compares the new UI with the old one and sends the browser **only the changed DOM parts**.
4. The server calls the API, just like Razor Pages. The browser never sees the API.

3.3 Blazor WebAssembly

In WebAssembly mode **the whole application is downloaded into the browser**: the .NET runtime (`.wasm`) and the `.dll` files of the app and its libraries. The same `.razor` components run in the browser:

1. nginx (or any other static file server) delivers `index.html`, `_framework/*.wasm` and `*.dll`. There is no .NET on the server.
2. The browser starts the .NET runtime and `Program.cs` with `WebAssemblyHostBuilder`.
3. The components run in the browser; clicks do not need the server.
4. The **browser** calls the API through `fetch()`. So the API address must be reachable from the browser (`localhost:6001`), and the API must allow **CORS**.

> **.NET 8 terminology.** Since .NET 8 the “Blazor Web App” template lets one application choose a **render mode**: Static Server, Interactive Server, Interactive WebAssembly or Interactive Auto (Server the first time, WebAssembly later). Our `SimpleStudentBlazorServer` uses Interactive Server for all pages, while `SimpleStudentBlazorWasm` is a separate (standalone) WebAssembly project, so the difference is clearly visible.

4. Comparison table

Criterion	Razor Pages	Blazor Server	Blazor WebAssembly
Where the UI C# code runs	On the server, per request	On the server, per browser tab (circuit)	In the browser (WebAssembly)
Page update	Whole page after every action	Only the changed parts	Only the changed parts
Connection to the server	Plain HTTP requests	Permanent SignalR (WebSocket) connection	Only file downloads and API requests
Who calls the API	Frontend server	Frontend server	Browser
API address	Internal: `http://simplestudentapi:8080`	Internal: `http://simplestudentapi:8080`	Public: `http://localhost:6001`
CORS on the API	Not needed	Not needed	**Required**
First load	Fast	Fast	Slower (a few MB of runtime), cached later
Interactivity without JavaScript	Limited (forms)	Full	Full
Server load	Only during a request	Memory for every connected user	Minimal (static files)
When the network drops	The next click works	UI stops, shows “reconnect”	UI works, only API requests fail
Secrets (keys, passwords)	Can stay on the server	Can stay on the server	**Not allowed**: `wwwroot` files and `.dll` are public
SEO / search engines	Excellent	Good with prerendering	Weak without prerendering
Hosting	ASP.NET Core server	ASP.NET Core server	Any static file server, CDN
Project SDK	`Microsoft.NET.Sdk.Web`	`Microsoft.NET.Sdk.Web`	`Microsoft.NET.Sdk.BlazorWebAssembly` + NuGet packages
Our project	`SimpleStudentFrontend` :6011	`SimpleStudentBlazorServer` :6031	`SimpleStudentBlazorWasm` :6041

5. The same task in three ways

Below are shortened but real code fragments from our projects. In the Blazor Server and WebAssembly projects **the page code is the same**; only `Program.cs`, the HTML shell and the way the `HttpClient` is created differ.

5.1 Registration in Program.cs

Razor Pages — SimpleStudentFrontend/Program.cs

```
builder.Services.AddRazorPages();
builder.Services.AddHttpClient<IStudentApiClient, StudentApiClient>(client =>
{
    var apiBaseUrl = builder.Configuration["ApiSettings:BaseUrl"] ?? "http://localhost:6001";
    client.BaseAddress = new Uri(apiBaseUrl);
});
...
app.MapRazorPages();
```

Blazor Server — SimpleStudentBlazorServer/Program.cs

```
builder.Services.AddRazorComponents()
                .AddInteractiveServerComponents();   // components run on the server
builder.Services.AddHttpClient();
builder.Services.AddScoped<ApiConnection>();         // one per browser tab (circuit)
...
app.MapRazorComponents<App>()
   .AddInteractiveServerRenderMode();
```

Blazor Server — Components/App.razor: render mode for all pages

```
<Routes @rendermode="new InteractiveServerRenderMode(prerender: false)" />
```

Blazor WebAssembly — SimpleStudentBlazorWasm/Program.cs

```
// this code runs IN THE BROWSER
var builder = WebAssemblyHostBuilder.CreateDefault(args);
builder.RootComponents.Add<App>("#app");     // <div id="app"> in wwwroot/index.html
builder.Services.AddScoped<ApiConnection>();
await builder.Build().RunAsync();
```

5.2 Loading the list of students

Razor Pages loads the data in `OnGetAsync()`, and the template renders it once. In a Blazor component the data is a field of the component: when the field changes, Blazor re-renders the related part of the HTML by itself.

Razor Pages — Pages/Students.cshtml.cs

```
public class StudentsModel : PageModel
{
    private readonly IStudentApiClient _api;
    public StudentsModel(IStudentApiClient api) => _api = api;

    public List<Student> Students { get; private set; } = new();

    public async Task OnGetAsync()
    {
        Students = (await _api.GetStudentsAsync()).ToList();   // GET /api/students
    }
}
```

Blazor (Server and WASM) — Pages/Students.razor

```
@page "/students"
@inject ApiConnection Api

<input class="search" @bind="_search" @bind:event="oninput" />
@foreach (var s in Filtered) { <tr><td>@s.Id</td><td>@s.FirstName</td>...</tr> }

@code {
    private List<Student> _students = new();
    private string _search = "";

    // recalculated after every key press – the API is not called
    private IEnumerable<Student> Filtered => _students.Where(s =>
        $"{s.FirstName} {s.LastName} {s.Email}"
            .Contains(_search, StringComparison.OrdinalIgnoreCase));

    protected override async Task OnInitializedAsync()
        => _students = (await Api.Client.GetStudentsAsync()).ToList();
}
```

Live search. `@bind:event="oninput"` updates `_search` after every key press, and `Filtered` filters the list that is already loaded. Razor Pages would need JavaScript or a new request with a `?search=` parameter.

5.3 Deleting a student

Razor Pages — Pages/Students.cshtml

```
<form method="post" asp-page-handler="Delete" onsubmit="return confirm('Delete?')">
    <input type="hidden" name="id" value="@s.Id" />
    <button type="submit" class="btn btn-delete">Delete</button>
</form>
```

Razor Pages — Pages/Students.cshtml.cs

```
public async Task<IActionResult> OnPostDeleteAsync(int id)
{
    await _api.DeleteStudentAsync(id);   // DELETE /api/students/{id}
    return RedirectToPage();             // the browser loads the whole list again
}
```

Blazor — Pages/Students.razor

```
@if (_confirmDeleteId == s.Id)
{
    <span>Delete?</span>
    <button @onclick="() => DeleteAsync(s.Id)">Yes</button>
    <button @onclick="() => _confirmDeleteId = null">No</button>
}
else
{
    <button @onclick="() => _confirmDeleteId = s.Id">Delete</button>
}

@code {
    private async Task DeleteAsync(int id)
    {
        await Api.Client.DeleteStudentAsync(id);
        _students.RemoveAll(s => s.Id == id);   // only this row disappears
    }
}
```

Deleting in Razor Pages needs a form, a `POST` request, a redirect and a new `GET` – three HTTP requests from the browser. In Blazor it is one C# method, and only one table row changes in the browser.

5.4 Form and validation

NSwag generated the `Student` class from `swagger.json` with the attributes `[Required]`, `[StringLength]` and `[Range]`. Blazor's `DataAnnotationsValidator` checks them while you type, so **the validation rules come from the API contract**. Errors returned by the API (400 ValidationProblem) are shown by Blazor next to the matching fields.

Razor Pages — Pages/StudentCreate.cshtml(.cs)

```
<form method="post">
    <input asp-for="Input.FirstName" required />
    <input asp-for="Input.EnrollmentYear" type="number" min="2000" max="2100" required />
    <button type="submit">Save Student</button>
</form>

[BindProperty] public StudentDto Input { get; set; } = new();

public async Task<IActionResult> OnPostAsync()
{
    try { await _api.CreateStudentAsync(Input.ToApi()); return RedirectToPage("/Students"); }
    catch (Exception ex) { ErrorMessage = ApiErrors.ToMessage(ex); return Page(); }
}
```

Blazor — Shared/StudentForm.razor (one form for create and edit)

```
<EditForm EditContext="_editContext" OnValidSubmit="SubmitAsync" novalidate>
    @* rules from the generated Student class *@
    <DataAnnotationsValidator />
    <InputText @bind-Value="Model.FirstName" />
    <ValidationMessage For="() => Model.FirstName" />
    ...
</EditForm>

@code {
    private async Task SubmitAsync()
    {
        try { await Save(Model); }        // Create... or UpdateStudentAsync
        catch (Exception ex)
        {
            // 400: { "errors": { "firstName": ["First name is required."] } }
            foreach (var (field, messages) in ApiErrors.FieldErrors(ex))
            {
                var property = char.ToUpperInvariant(field[0]) + field[1..];
                _serverErrors.Add(_editContext.Field(property), messages);
            }
            _editContext.NotifyValidationStateChanged();
        }
    }
}
```

Pitfall: novalidate. A field with `type="email"` turns on the browser's HTML5 validation: with the value “x” the form is simply not submitted, and neither Blazor nor the API shows an error. The `novalidate` attribute on `EditForm` turns off the browser check, so Blazor and the API show the errors. We found this bug while testing the example.

5.5 API address and HttpClient

The most important practical difference is **who sends the HTTP request**. The server-side frontends use the internal name inside the Docker network; WebAssembly uses the address the browser can see.

Blazor Server — docker-compose.yml

```
Backends__0__BaseUrl: http://simplestudentapi:8080   # server → API (Docker network)
Backends__0__PublicUrl: http://localhost:6001        # browser → Swagger link
```

Blazor WebAssembly — wwwroot/appsettings.json (a public file!)

```
{ "Name": ".NET", "Url": "http://localhost:6001" }
```

Services/ApiConnection.cs in both Blazor projects

```
// Blazor Server: HttpClient from IHttpClientFactory on the server
var http = _httpClientFactory.CreateClient();
http.BaseAddress = new Uri(backend.BaseUrl);

// Blazor WebAssembly: HttpClient = the browser's fetch()
new StudentApiClient(new HttpClient { BaseAddress = new Uri(Current.Url) });
```

6. CORS: why only WebAssembly needs it

CORS (Cross-Origin Resource Sharing) is a browser security rule: JavaScript (and WebAssembly) loaded from one address (http://localhost:6041) may read a response from another address (http://localhost:6001) only if the latter allows it with Access-Control-Allow-* headers.

- In Razor Pages and Blazor Server the **server** calls the API. CORS rules do not apply to servers, so the API needs nothing extra.
- In Blazor WebAssembly the **browser** calls the API. Before a POST/PUT with JSON the browser sends a **preflight** OPTIONS request; if the API does not answer with the right headers, the request is blocked.
- The PHP API had CORS from the start (Http::cors()), the Python API has CORSMiddleware, and the .NET API was extended with the code below.

.NET API — SimpleStudentApi/Program.cs

```
builder.Services.AddCors(options =>
    options.AddDefaultPolicy(policy => policy
        .AllowAnyOrigin()
        .AllowAnyHeader()
        .AllowAnyMethod()
        .WithExposedHeaders("Location")));   // the browser can read the 201 Location header
...
app.UseCors();
```

> **In production.** AllowAnyOrigin() is fine for a teaching project. A real public API should list the allowed addresses, e.g. WithOrigins("https://students.example.com").

7. When to choose which

Choose	When…	Be careful if…
Razor Pages	Content website, forms, admin pages; SEO matters; you need the simplest and most robust solution.	You need a lot of interactivity without reloads – you would have to write JavaScript.
Blazor Server	Internal business application; lots of interactivity; fast first load; the API and DB are in a private network; the code must stay on the server.	Many users are connected at the same time (memory on the server), or the network is unstable (latency, disconnects).
Blazor WebAssembly	The app must keep working when the connection drops (PWA); hosting as static files/CDN; you want to move work to the client.	Large first download; all .dll files and settings are public; the API must be public, with CORS and authentication (tokens).

In a heterogeneous system there is no single right answer: in our lab three .NET frontends, a PHP frontend and a Flutter app use the same API. The decision depends on the users, the network and the hosting – not on the API.

8. Common problems and fixes

Problem	Where	Cause and fix
The API is called twice when a page opens	Blazor Server	Prerendering: the page is rendered on the server first, then again interactively. Turned off in our example: InteractiveServerRenderMode(prerender: false).

Problem	Where	Cause and fix
“Attempting to reconnect to the server…“	Blazor Server	The SignalR connection dropped (container restarted, network). The page reconnects by itself; the circuit state is lost.
Clicking “Save” does nothing	Blazor	The browser's HTML5 validation (`type="email"`) blocks the form. Add `novalidate` to `EditForm`.
“TypeError: Failed to fetch”	Blazor WASM	The API is not running or does not allow CORS. F12 → Console shows the missing header.
The address `simplestudentapi:8080` does not work	Blazor WASM	That name exists only inside the Docker network. The browser needs `localhost:6001`.
A changed `wwwroot/appsettings.json` has no effect	Blazor WASM	The browser cached it: Ctrl+F5 (Cmd+Shift+R).
The API selection is lost after F5	Blazor Server	Expected: the selection lives in the circuit state, and a reload creates a new circuit.

9. Running the projects

From the folder `student-docker-lab-plus-mssql` start SQL Server (once) and all services:

Terminal

```
docker-compose -f docker-compose.infra.yml up -d
docker-compose up --build
```

Address	What it is
http://localhost:6011	Razor Pages frontend
http://localhost:6031	Blazor Server frontend
http://localhost:6041	Blazor WebAssembly frontend
http://localhost:6021	PHP frontend
http://localhost:6001/swagger · :6002 · :6003	.NET · PHP · Python API with Swagger UI

The `README.md` (EN) and `README_LT.md` of every project also describe running without Docker (`dotnet run`).

10. Practical tasks

1. Run all three .NET frontends. In each one open F12 → **Network** and delete one student. How many requests did the browser send, and to which addresses? Explain the difference.
2. While the Blazor Server page is open, stop the container `simplestudentblazorserver` (`docker stop simplestudentblazorserver`). What happens to the page? Repeat with `simplestudentblazorwasm`.
3. Temporarily remove `app.UseCors();` from the .NET API, rebuild the API and open the WebAssembly frontend. Which error does the Console show? Why do Razor Pages and Blazor Server keep working?
4. In Blazor `Students.razor` add sorting by last name (click on the column header) without a new API request.
5. In the Razor Pages frontend create a **Faculty** page (`GET /api/faculty`) like Blazor's `Faculty.razor`. Compare how much code the expanding groups needed.

6. Switch Blazor to the PHP API and try to save a student with the email “x”. Where did the error message come from: the browser, Blazor or the API?

11. References

- ASP.NET Core Razor Pages: https://learn.microsoft.com/aspnet/core/razor-pages/
- ASP.NET Core Blazor: https://learn.microsoft.com/aspnet/core/blazor/
- Blazor render modes: https://learn.microsoft.com/aspnet/core/blazor/components/render-modes
- Blazor hosting models: https://learn.microsoft.com/aspnet/core/blazor/hosting-models
- CORS in ASP.NET Core: https://learn.microsoft.com/aspnet/core/security/cors
- NSwag (OpenAPI → C# client): https://github.com/RicoSuter/NSwag
- Course projects: `code-examples/student-docker-lab-plus-mssql` (README.md / README_LT.md in every project)